Automated Rating Review System

1. Project Overview

An automated review system analyzes customer reviews and predicts the corresponding star ratings. Natural language processing (NLP) is used to extract sentiment and key features from the review texts.

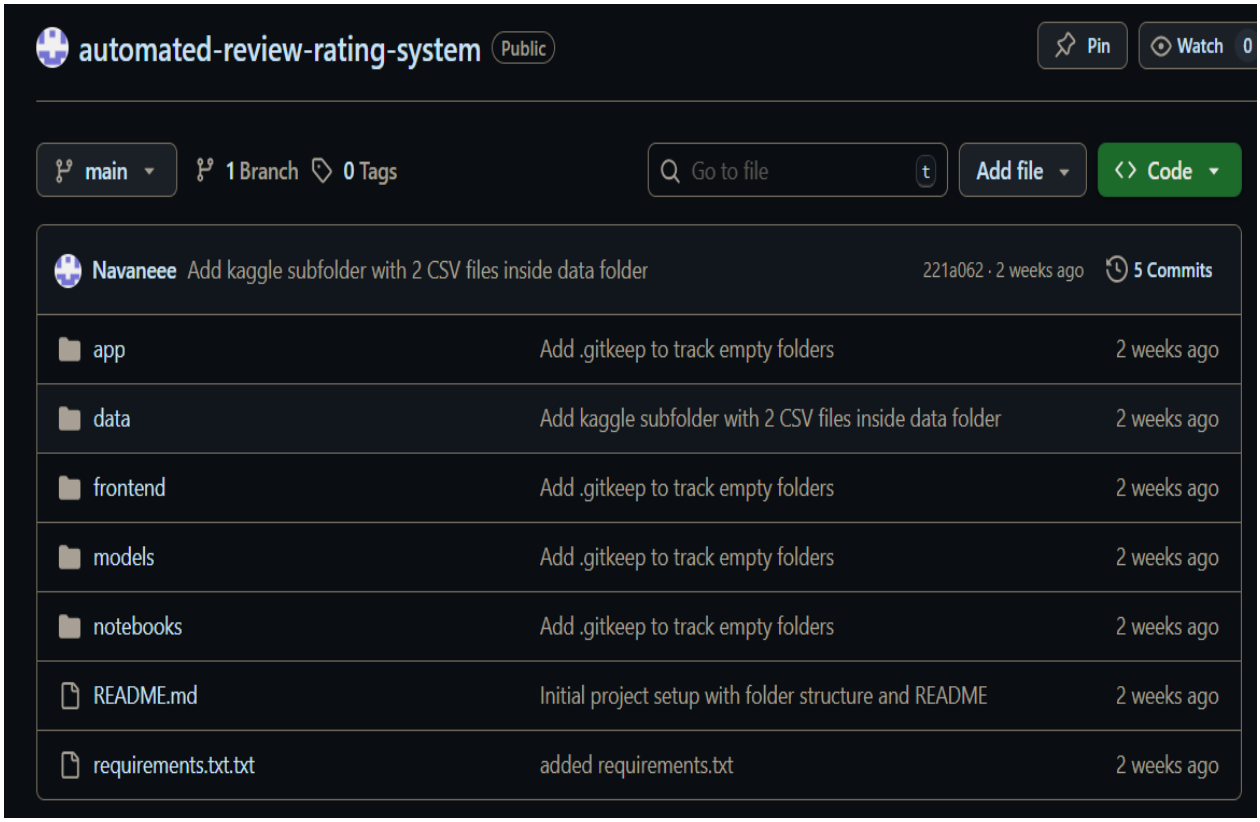
2. Environment Setup

- Installed Python 3.13.3 and the required libraries such as pandas, numpy, matplotlib, seaborn, and scikit-learn. For data preprocessing, NLP, and visualization, these packages were utilized.
- VS Code is used as the IDE.

3. GitHub Project Setup

Created a GitHub respository named automated-review-rating-system.

Structure of directory.



4. Data Collection

The dataset was collected from Kaggle and it consists of 568454 Amazon product ids, customer reviews, star ratings and 7 more features also.

Dataset link : https://github.com/Navaneeee/automated-review-rating-system/tree/main/data/kaggle/review_data.csv

5. Data Preprocessing.

5.1 Data Preview and Dropping Unnecessary Columns

The Amazon review data consisted of 10 different columns. It preprocessed to include only review ratings and review text. All other columns were removed. The preview is given below. (568454, 2) is the dimension of the new data.

```
data.head()
```

	rating	review_text
0	5	I have bought several of the Vitality canned d...
1	1	Product arrived labeled as Jumbo Salted Peanut...
2	4	This is a confection that has been around a fe...
3	2	If you are looking for the secret ingredient i...
4	5	Great taffy at a great price. There was a wid...

5.2 Missing Values

There were no missing values present in the data.

5.3 Duplicate Values Handling

There are 174779 number of duplicate rows were present in the data and all of them were removed with the code

```
data.drop_duplicates(inplace=True)
Also 96 duplicate reviews were present in review text column.Those were also
removed with the code.
data.drop_duplicates(subset=['review_text'], inplace=True)
```

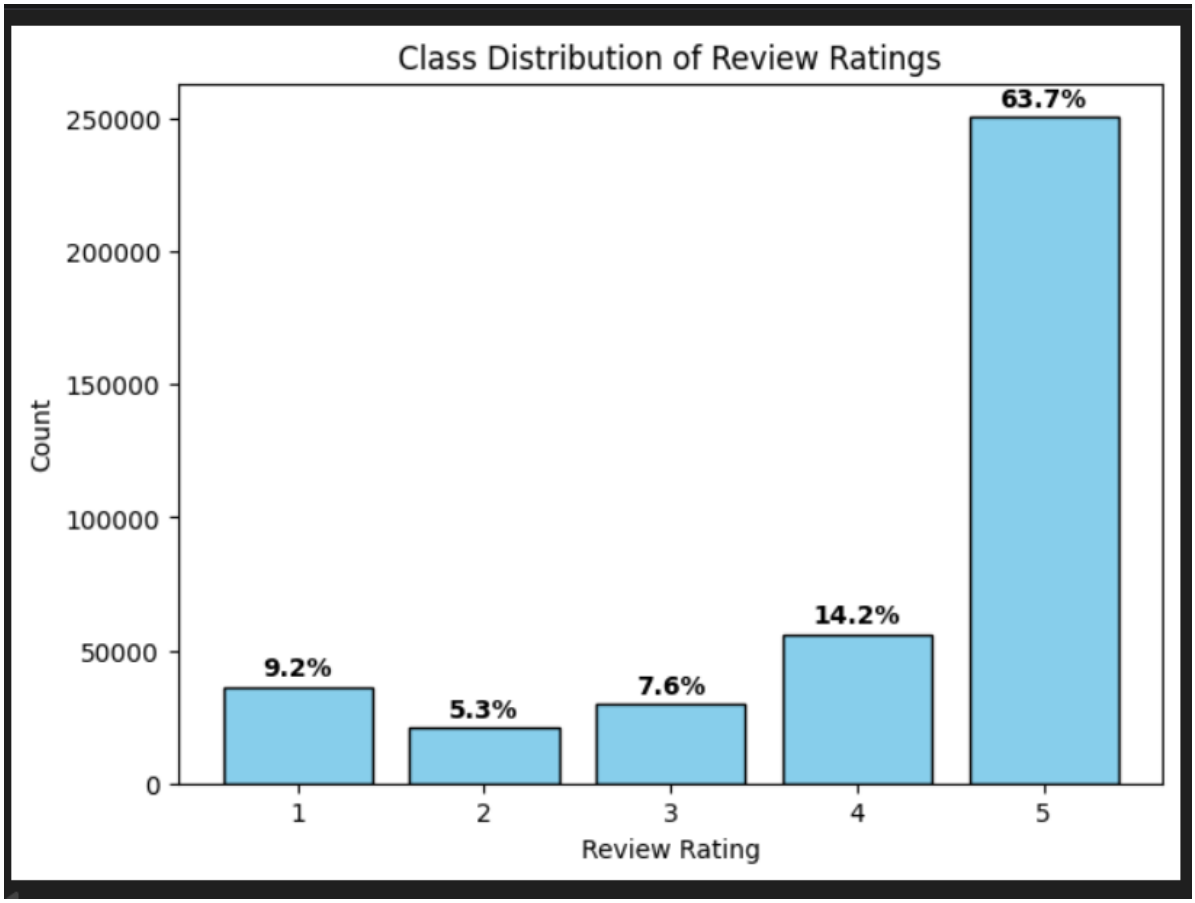
The final shape of the data was (393579, 2)

5.4 Class Distribution

The total value count of each rating is found with using value_count() method and it shown as

```
Class Distribution (Counts):
rating
1     36275
2     20792
3     29754
4     56042
5    250716
Name: count, dtype: int64
```

The distribution of ratings is then plotted with a barplot.



The inferences from the barplot were;

1. The dataset is heavily imbalanced.
2. 5-star reviews dominate and it's around 64%
3. 2-star reviews are very low, around 5%
4. To make the data balace,we try random downsampling to the high ratings, 250,716 ratings are reduced to 50,000 and all other ratings are increased to 50,000.

5.5 Splitting the dataset to Balaced data and Imbalaced data.

The whole data is spliied into a balanced data with the code

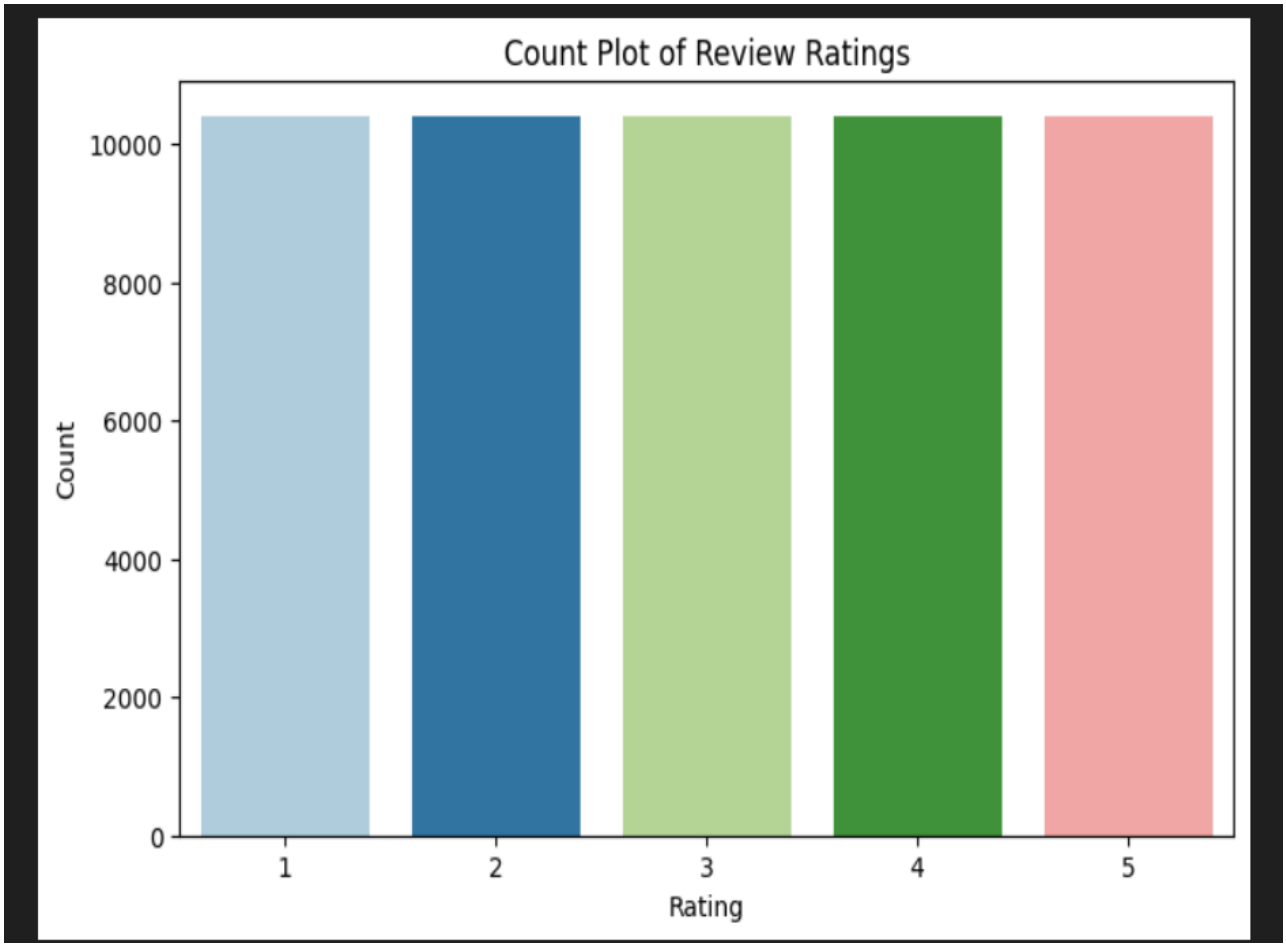
```
#define the sample size(half of class 2 star ratings' count)
sample_size=class_counts[2]//2

#Sample from rating 2
#Selecting all the rows where rating column =1 and sample size is half of lowest
count class
df2_list=data[data['rating']==2].sample(n=sample_size,random_state=42) #ensures
the sampling is reproducible

#Sample from rating 1,3,4,5
#selecting same number of sample size rows from all other ratings' class
dfr_list=[data[data['rating']==r].sample(n=sample_size,random_state=42) for r in
[1,3,4,5]]

#concatenate df2_list with all others( df2,[df1,df3,df4,df5])
balanced_dataset = pd.concat([df2_list] + dfr_list)

#Create second dataset consising of remaining rows
second_dataset = data.drop(balanced_dataset.index)

#Shuffle the combined dataset
balanced_dataset = balanced_dataset.sample(frac=1,
random_state=42).reset_index(drop=True)


#Saving the balanced dataset.
balanced_dataset.to_csv(r"C:\Users\kjnav\Documents\Navaneee\automated-review-
rating-system\data\cleaned\balanced_data.csv", index=False)
```

And splitted into an imabalanced dataset with code;

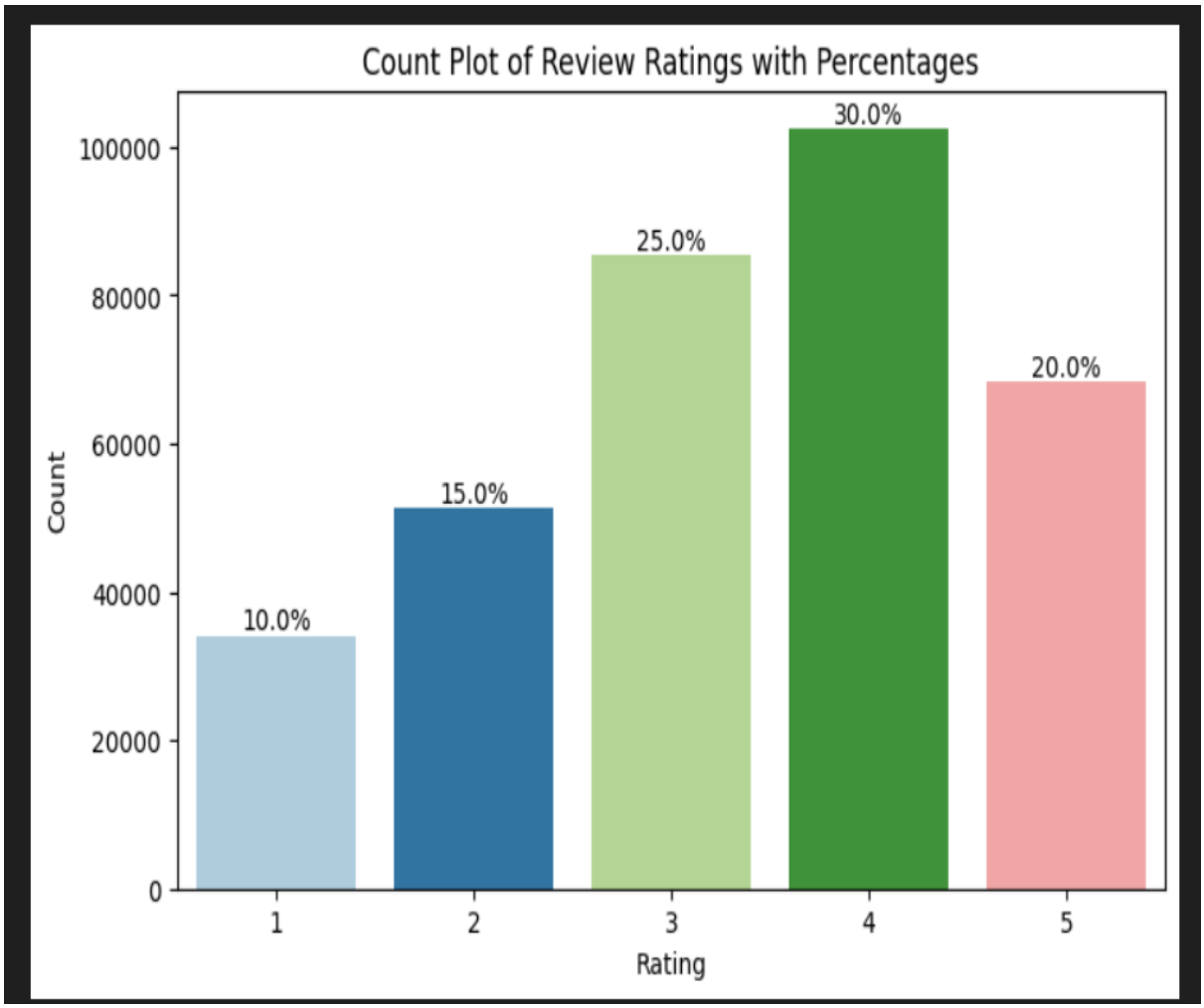
```
#setting the target percentages
percentage_dict={1:0.10,          #  1 *      =10%
                 2:0.15,          #  2 **     =15%
                 3:0.25,          #  3 ***    =25%
                 4:0.30,          #  4 ****   =30%
                 5:0.20}          #  5 *****  =20%

#total number of rows in the dataset
total=len(second_dataset)

#create an empty list to keep new data
new=[]

#for each ratings,we take number of rows we want according to the target
percentage
for ratings,percentage in percentage_dict.items():
    total_rows_needed=int(percentage*total)
```

```
    rows=second_dataset[second_dataset['rating']==ratings]
    #select only that rating(filtering the rows)
    sample_rows=rows.sample(total_rows_needed,replace=True,random_state=42)
    new.append(sample_rows)

Thereafter saved,naming imbalanced_data

#save this  imbalnced dataset.
imbalanced_data.to_csv(r"C:\Users\kjnav\Documents\Navaneee\automated-review
rating-system\data\cleaned\imbalanced_data.csv", index=False)
```

The countplot of balanced data;



The countplot of imbalanced data



6. Balanced Dataset

Dataset link : https://github.com/Navaneee/automated-review-rating-system/tree/main/data/kaggle/balanced_data.csv

6.1 Data Import and Preprocessing

This stage involves importing the dataset, inspecting its structure, handling missing values, and performing initial cleaning steps such as removing duplicates and computing word counts.

Data Loading

```
data=pd.read_csv(r"C:\Users\kjnav\Documents\Navaneee\automated-review-rating-system\data\cleaned\balanced_data.csv")
```

The preview of the data just given.

	rating	review_text
0	5	Love this product, however the Amazon price is...
1	4	This has recently become my staple breakfast c...
2	4	This spread is just right but costs only about...
3	2	I guess I learned from this purchase that rais...
4	2	This coffee is sort of grocery store quality-a...

Importing all the necessary libraries

The following Python libraries were used for text preprocessing, feature extraction, and dataset splitting.

Library	Purpose
string	Contains string constants and utility functions. Used to remove punctuation from text.
nltk	Natural Language Toolkit for NLP tasks such as tokenization, stemming, and lemmatization.
from nltk.corpus import stopwords	Provides a list of common stopwords (e.g., “the”, “is”, “and”), which are removed to reduce noise.

Library	Purpose
from nltk.stem import WordNetLemmatizer	Reduces words to their base or dictionary form (lemmatization).
from sklearn.model_selection import train_test_split	Splits datasets into training and testing sets for model evaluation.
from sklearn.feature_extraction.text import TfidfVectorizer	Converts textual data into numerical features using TF-IDF, representing the importance of words across documents.

6.2 Text Preprocessing

Created a user defined function called “preprocess_text”. This function cleans and prepares the review text for analysis using Natural Language Processing (NLP) techniques.

1. **Lowercasing:**
   Converts all text to lowercase to maintain consistency.
2. **Removing URLs and HTML tags:**
   Eliminates links and HTML elements that are not useful for text analysis.
3. **Removing emojis and special characters:**
   Keeps only alphabetic characters to focus on meaningful words.
4. **Tokenization:**
   Splits the text into individual words (tokens).
5. **Stopword Removal:**
   Commonly used words that do not contribute much to the meaning or sentiment of a sentence are removed. The list of stopwords used comes from **NLTK’s** default English stopword set. Removing these words helps reduce noise and improves the focus on meaningful terms for tasks like text classification, sentiment analysis, or information retrieval. **NLTK** provides a simple and lightweight stopword list ready to use, making it ideal for quick preprocessing in research or small projects.

   spaCy also offers stopword lists, but it is more resource-intensive and geared toward full-featured NLP pipelines. For tasks where only stopword removal is required, NLTK is faster and easier to implement. The list of stopwords used comes from NLTK’s default English stopword set, which includes words such as:

{'by', 'yourself', 've', 'been', 'other', 'in', "didn't", 'each', 'this',"he'll", 'am', "they've", "you're", 'ourselves', 'ours', 'doing', "it'll","shan't", 'hadn', 'most', 'than', 'they', "we'd", 'she', 'haven',"couldn't","hasn't", 'his', 'out', 'over', "we'll", "isn't", 'against', 'which', 'hers',

'or', "won't", "you'd", 'above', 'should', 'on', 'because', 'he', "she'll",'does', 'couldn', 'shan', "don't", "they'd", 'with', 'a', 'why', 'weren','shouldn', 'yourselves', 'ain', "she'd", 'more', 'll', 'myself', 'theirs','but', 'about', 'them', 'did', 'to', 'has', 'be', 'so', 'of', 'too', 'while',"should've",'those', 'don', "she's", 'some', 'all', 'such', "aren't", 'from','here', 'whom', "mightn't", 'wouldn', 'm', "you've", 'at', 'themselves','mustn', 'if', 'then', 'now', 'nor', 'that', "they're", 'won', 'no', 'have','me', 'were', 'own', "we've", 'any', 'o', 'do', 'mightn', "we're", "he's","shouldn't", 'further', 'was', 'there', 'is', 'its', 'between', 'for', 'only','an', 'my', 'i', "haven't", 'where', "hadn't", 't', 'what', 'into', 'once','below', 'himself', 'having', 'are', 'ma', 'off', 'we', 'who', 'before','yours', 'hasn', 'not', 's', 'as', 'doesn', 'didn', 'him', "that'll", 're',"it'd", 'isn', 'it', 'can', 'her', 'd', "weren't", 'had', 'itself', 'their','herself', "you'll", 'again', 'you', 'down','your', "i'd", "it's", 'through','under', 'very', 'wasn', 'during', 'how', "wasn't", "mustn't", "i've", 'both', 'until', 'y', "i'm", 'and', 'the', 'after', 'when', 'few', 'aren', 'our',"they'll", 'up', 'being', "i'll", 'needn', 'these', 'same', 'will',"wouldn't", "he'd", 'just', "doesn't", "needn't"}

6. Lemmatization

Lemmatization is the process of converting each word to its base or dictionary form (known as the lemma).For example:

“walking” → “walk”,

“better” → “good”.

This step helps ensure that words with similar meanings are treated as the same during analysis, thereby improving model consistency and reducing feature sparsity. While both **stemming** and lemmatization aim to reduce words to their root form, lemmatization is more accurate and linguistically meaningful.

7. Rejoining:

The cleaned tokens are combined back into a single string. The cleaned text is saved in a new column “cleaned_review”, and a “word_count” column is created to store the number of words in each processed review.

8. Removed too long or short texts:

The review texts above word below 3 and above 175 is removed. The codes given below.

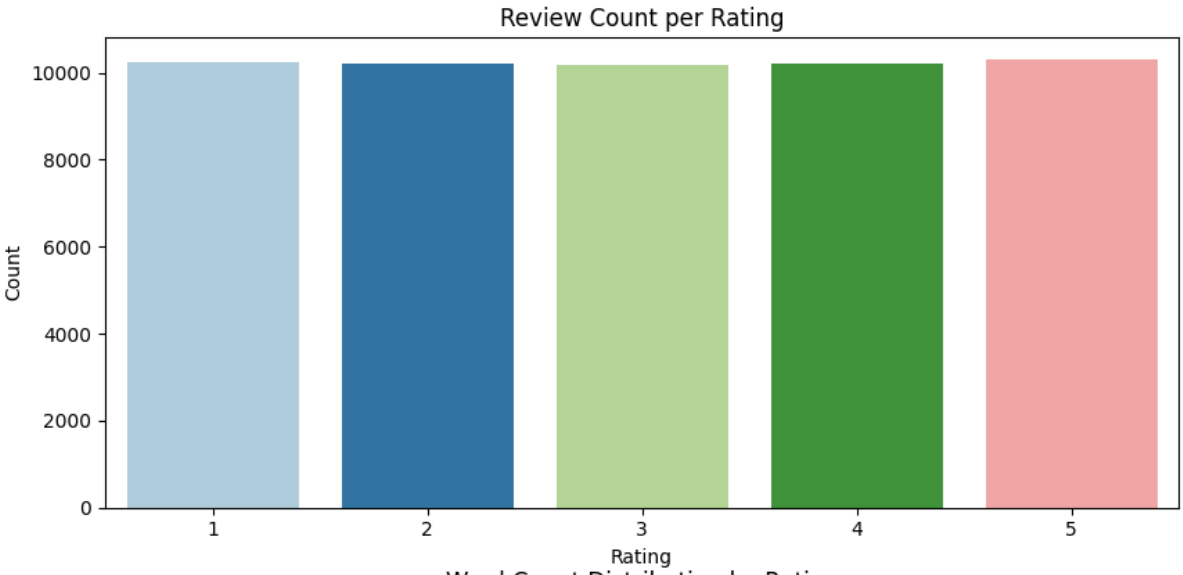
```
stop_words = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def preprocess(text):

    # lowercase
    text = text.lower()

    # remove URLs
    text = re.sub(r"http\S+|www\S+", "", text)

    # remove HTML tags
    text = re.sub(r"<.*?>", "", text)
```

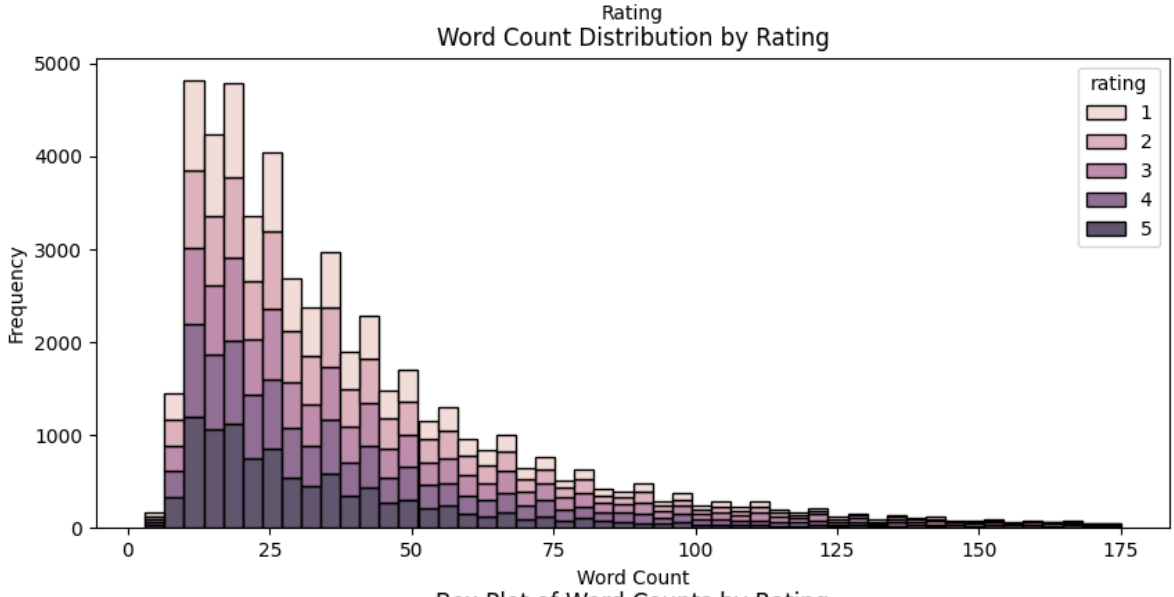
```
    # remove emojis and non-alphabetic characters
    text = re.sub(r"[^a-z\s]", "", text)

    # tokenize
    tokens = text.split()

    # remove stopwords & lemmatize
       tokens = [lemmatizer.lemmatize(word) for word in tokens if word not in
       stop_words]

    # rejoin
    return " ".join(tokens)

# Applying preprocess function to review text column.
data["cleaned_review"] = data["review_text"].astype(str).apply(preprocess)

#finding count of words in review text
data["word_count"] = data["cleaned_review"].apply(lambda x: len(x.split()))
```

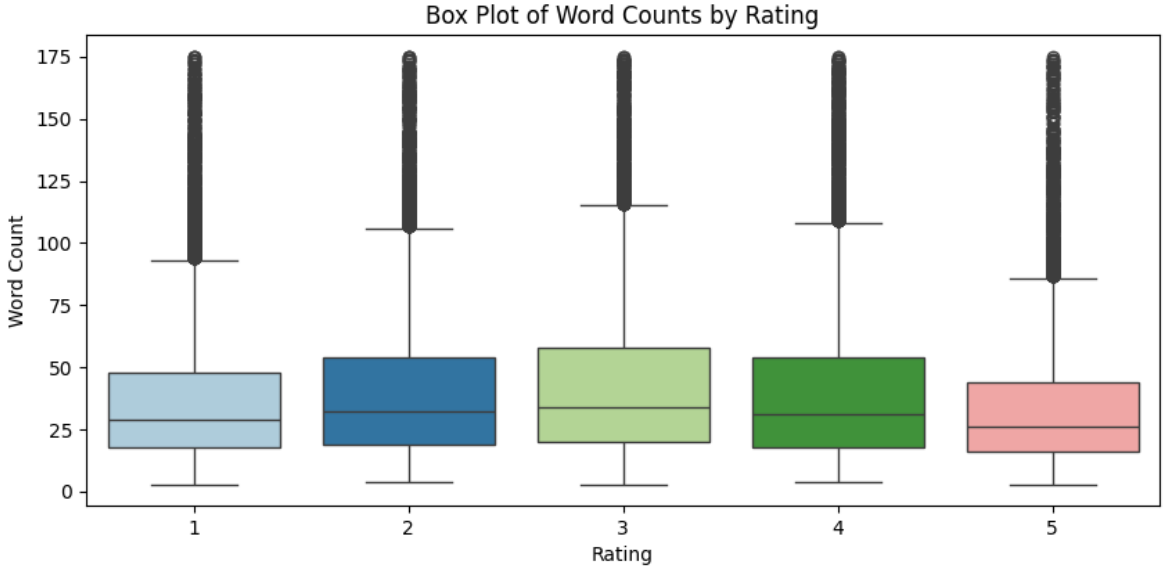
6.3 Data Visualization

A subplot consisting of a barplot ,histogram and box[plot of word count by ratings ploted.

```
#subplots of 3 rows and 1 column
fig, axes = plt.subplots(3, 1, figsize=(10, 15))

#bar plot: Review count per rating
sns.countplot(x="rating", data=data, palette="Paired", ax=axes[0])
axes[0].set_title("Review Count per Rating")
axes[0].set_xlabel("Rating")
axes[0].set_ylabel("Count")

#histogram of word counts
sns.histplot(data, x="word_count", hue="rating", multiple="stack", bins=50,
ax=axes[1])
axes[1].set_title("Word Count Distribution by Rating")
axes[1].set_xlabel("Word Count")
axes[1].set_ylabel("Frequency")

#box plot of word counts
sns.boxplot(x="rating", y="word_count", data=data, palette="Paired", ax=axes[2])
axes[2].set_title("Box Plot of Word Counts by Rating")
axes[2].set_xlabel("Rating")
axes[2].set_ylabel("Word Count")

plt.tight_layout()
plt.show()
```


Review Count per Rating
Count
Rating
0
2000
4000
6000
8000
10000
1
2
3
4
5


Word Count Distribution by Rating
Frequency
Word Count
rating
1
2
3
4
5
0
1000
2000
3000
4000
5000
0
25
50
75
100
125
150
175


Box Plot of Word Counts by Rating
Word Count
Rating
0
25
50
75
100
125
150
175
1
2
3
4
5

The inferences from the plots are;

1. Barplot
    a. The bar heights are same, means balanced the dataset across all ratings from 1 to 5.
2. Histplot
    a. Most reviews have fewer words and around 50.The distribution is right-skewed,large number of short reviews, and fewer long ones.
    b. The overlap between ratings shows review length is not a strong indicator of sentiment/rating.Both good and bad reviews can be short or long.
3. Boxplot
    a. Median word counts are around 25–35 words
    b. The interquartile ranges are also similar, meaning review length doesn't vary much between 1-star and 5-star ratings.
    c. Many outliers exist (long reviews above 100 words). These could be detailed opinions or complaints.
    d. Slightly shorter reviews appear in extreme ratings (1 and 5), while middle ratings (2 to 4) show more spread ,possibly reflecting more nuanced feedback.
    e. Outliers which are long reviews may carry rich detail, potentially useful for deeper analysis of sentiment.

6.4 Sample ratings preview

5 Sample reviews of 5 ratings are printed with the code

```
# preview of sample reviews per rating
for r in sorted(data["rating"].unique()):
    print(f"sample review of rating {r}")
    display(data[data["rating"] == r]["cleaned_review"].sample(min(5,
len(data[data["rating"] == r]))).to_list())
```

sample review of rating 1

1.'wondering much cheaper flavor know cat wouldnt touch acted like stunk backed awayhe went eat day old dried cat food rather eat junkim concerned someone wrote might tainted recalled later well ill save case get recalledmaybe get refund dont like amazon wont let return food itemsunless recalled especially unopened cansalot people gave good rating alot people didnt think one cat like odds bad shouldnt buy',

2. 'ordered box newman dried apricot understood dark color lack preservative ate anyway became illvomiting nausea chill dizzinessabout hour eating occur apricot thougnt bad virus

soi ate slow learnersame result miserable contacted amazon gracious refunding moneythe remainder apricot trash warned',

3. 'already ordered directly cafe britt jumped chance order cafe britt product amazon thus saving shipping cost even getting break price experience dealing cafe britt great ordered lb bag whole bean costa rica dark roast coffee loved wanting try something different ordered lb bag tres rio valdivia whole bean coffee thru amazomcom upon opening first bag noticed difference quality quite apparent assumed probably different type coffee opening second

4.bag think looking freshness date packaging ordered coffee july received august rd package freshness date jun lot appears amazon shipped coffee way date know local grocery store going sell food date amazon itsecond entryhaving spoken customer service rep amazon told outofdate product replaced cost appreciate must commend amazon responsiveness complaint',

5. 'ordered product received broken bottle amazon excellent reshipped entire order one day shipping unfortunately even cent bottle overpaid cheese taste really salty mess bottle average product save money'

sample review of rating 2

1.'parachute coconut oil really coconut kind hard tell coconut oil smell like sort bacon fat tried use dont like smell bacon product totally bad',

2. 'mom turned alba milkshake remember tasting pretty good thrilled find amazon year ordered item massive quantity im trying find people give since cant force fake aftertasteladen stuff throat dont know different ingredient faulty memory stuff terribly disappointing borderline disgusting',

3.'dog loved taste dog food didnt agree system month old lab puppy amount food recommended much food dog think stuff expensive also',

4. 'coffee bean purchased directly roaster typically arrive hour time roasted always ship within hour roasted inexpensive single sourced precisely roasted bean brewed correctly produce unbeatable cup coffeewhen purchased amazon unfortunately bean arrived day roasted date day roasted printed bag format dayoftheyearyear bag amazon came printed meaning st day june today august shipped day agocoffee peak moment come roaster decline every minute via dissipation volatile aromatics oxidation staling buy directly roaster want fresh buy amazon want cheap mediocre halfstale coffee bulk',

5. 'listing say milk chocolate covered however received dark chocolate covered peanut really dont like dark chocolate much ended giving away part ate theyre alright listed']

sample review of rating 3

1.'ingredient dog food look pretty good suspect thing like corn gluten meal animal digest ew food coloring cant recommend liked made poop often runny going back proplan',

2. 'product arrived time correct flavor type problem though opened brown shipping box looked like whoever shipped opened product took one pouch pouch pack kind would set grocery store also ordered different flavor time company box also missing pouch total screwed pouch gum didnt bother sending product back complaining let face trouble wasnt worth gum worth still kind disappointing someone would steal customer',

3.'area many option instant oatmeal gluten free one one option ok however one need add cinnamon',

4. 'bought friend dont know able make yet really careful eatsonly gluten free food',

5. 'meal tasty filling quite healthy berry really compliment crunchy spice rice however easy take onthego might think water need boiling added pilaf need another container heat water always easy find youre taking meal go found work well use hot water water cooler coffee pot'

sample review of rating 4

1.'im bit licorice fanatic obviously else buy case say good mass produced licorice better specialty shop price everyday snacking good licoricejust dont know licorice great tasking candy also good settling nerve sometimes itll make upset stomach little less upset maybe psychosomatic seem work',

2. 'cat loved liver treat packaging tore shipping patch packet',

3.'taste good difficult dissolve even cold water need stirredwhisked quite powder dissolve best dissolved cold water clump even water lukewarm',

4. 'family truly delight eating jyoti saag delicious satisfying part meal id feel better however feeding jyoti saag family reason believe can dont leach bpa saag',

5. 'great mix used many baking item pancake muffin turn good simple make also water added rather milk especially easy use']

sample review of rating 5

1.'officially new shampoo conditioner combo smell absolutely incredibleso fresh clean husband always compliment nice hair smell soft make hair shiny manageable rotate shampoo every day keep hair drying week use antidandruff shampoo condition every day dont use lot heat styling put product hair finding shampoo conditioner let wash hair air dry leaving soft laying nicely smelling great huge hair color treated prefer way length conditioner feel hair',

2. 'purchased sister holiday loved generous portion wonderful variety especially hard time thoughtful gift send love service excellent way delivery thank',

3. 'lingonberry great rotisserie chicken almost impossible fine supermarket even whole food doesnt brand consissently great quality',

4.'marmite aquired taste slightly salty yeasty spread use toast sandwich supposed good people eat like tastesmarmite isnt easy find u get larger health food type chain like sun harvest farm whole food market charge g jar one would consider charged shipping well product price deciding whether ordering marmite online value',

5. 'product calorie low one would expect icy grainy dessert opposite fact smooth creamy flavor taste greatwe kind thing arctic zero make treat rival ice creamextra peanut buttery chocolatemix tablespoon pb little splenda enough hot water make saucy consistancy pour chocolate chocolate peanut butter arctic zero heavenpistachiocrush cup shelled pistachio piece mixin along drop almond extract vanilla arctic zero yumtropical strawberryheat crushed pineapple little splenda pour strawberry arctic zerotropical iiheat cup apple juice tablespoon splenda teaspoon cornstarch add sliced banana disk warm sauce thickened pour strawberry vanilla chocolate chocolate peanut butter arctic zerocrispy chocolate mintslightly crush packet quaker calorie pack chocolatey mint mini delight mixin vanilla chocolate alpine zerocherry jubileechop cup maraschino cherry warm juice pour mixin vanilla strawberry arctic zerothese favorite come ton eating arctic zero plain great'

7. Train Test Split

The dataset is split into training (80%) and testing (20%) sets using the train_test_split() function.

The stratify=y parameter ensures that the proportion of each rating category remains the same in both the training and test sets.

```
x = balanced["cleaned_review"] # training
y = balanced["rating"]    # testing
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
stratify=y, random_state=42)
```

8. TF-IDF Vectorization

TF-IDF is a statistical measure that evaluates how important a word is in a document relative to a collection of documents (corpus). It combines two components:

- Term Frequency (TF): Measures how often a word appears in a document.

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

- Inverse Document Frequency (IDF): Measures how unique or rare a word is across all documents.

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{Number of documents containing term } t}$$

- TF-IDF Score:

$$TFIDF(t, d) = TF(t, d) \times IDF(t)$$

- Words that occur frequently in a document but rarely across all documents get higher TF-IDF scores, highlighting their importance for classification.Common words like "the" or "is" get low scores because they appear in almost all documents.
- This TF-IDF representation allows the model to focus on the most relevant words for predicting review ratings

```
vectorizer = TfidfVectorizer(max_features=5000)
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("TF-IDF Train shape:", X_train_tfidf.shape)
print("TF-IDF Test shape:", X_test_tfidf.shape)


#max_features=5000 limits the number of features to the top 5000 most important
words in the dataset.#fit_transform() learns the vocabulary from the training data
and converts it into TF-IDF features.
#transform() converts the test data into the same feature space without learning a
new vocabulary.
```

the textual review data is converted into numerical features using “TF-IDF (Term Frequency-Inverse Document Frequency**)”**. Machine learning models require numerical input, so text data must be transformed into a numerical representation.